Login, Registration & Logout with C# and SQL Server

Student ID: 24-59219-3 | Project: 59219_LoginSystem

1. Introduction

This project is a Windows Forms login system developed with C# and SQL Server. It contains Login, Register, and Home forms and implements registration, login, logout, password hashing, failed-login handling, a searchable user grid, and a SQL injection demonstration comparing vulnerable and fixed data-access code.

2. Environment

Student ID: 24-59219-3

SQL Server: LocalDB ((localdb)\MSSQLLocalDB)

Visual Studio: 2022 Community

Target Framework: .NET Framework 4.7.2

Connection string (App.config):

```
Data Source=(localdb)\MSSQLLocalDB;Initial Catalog=ID2459219_LoginDB;
Integrated Security=True;TrustServerCertificate=True
```

No real password is included in the connection string or the submitted repository, since LocalDB uses integrated Windows authentication.

3. Database and Schema.sql

The database and Users table were created using SQL Server Object Explorer inside Visual Studio, by running a New Query against the LocalDB connection. The full script is saved in Schema.sql. It creates the ID2459219_LoginDB database and the dbo.Users table with six columns:

- UserID — INT, IDENTITY(1,1), PRIMARY KEY
- Username — NVARCHAR(50), NOT NULL, UNIQUE
- PasswordHash — NVARCHAR(200), NOT NULL
- Email — NVARCHAR(100), NULL
- FullName — NVARCHAR(100), NULL
- CreatedAt — DATETIME, DEFAULT GETDATE()

4. Registration

RegisterForm.btnRegister_Click first validates the input: no empty fields, password at least 6 characters, password and confirm-password match, and the email contains "@". It then calls DatabaseHelper.UsernameExists(), which runs a parameterized SELECT COUNT(*) query through ExecuteScalar() to check for a duplicate username. If the username is free, DatabaseHelper.RegisterUser() hashes the password with HashPassword() and inserts the new user with a parameterized INSERT through ExecuteNonQuery(). On success, a confirmation message is shown, the form is cleared, and the user is returned to the Login screen.

5. Login

LoginForm.btnLogin_Click takes the entered username and password and calls DatabaseHelper.ValidateLogin(). This method runs a parameterized SELECT (using a SqlDataReader) to fetch the stored PasswordHash and FullName for that username, hashes the entered password with the same SHA-256 routine, and compares the two hashes. If they match, LoginSuccess() opens HomeForm and passes the FullName through. If not, LoginFailed() increments a failedAttempts counter and shows the number of attempts remaining; after 3 failed attempts the Login button is disabled.

6. Logout

HomeForm.btnLogout_Click simply calls this.Close(). LoginForm attaches a FormClosed event handler to the HomeForm instance before showing it, so when HomeForm closes, LoginForm.ClearForm() runs — clearing the textboxes and focusing the username field — and the Login form is shown again. The application never calls Application.Exit() on logout, and HomeForm is properly closed (not just hidden), so no orphan forms are left running.

7. Password Hashing

DatabaseHelper.HashPassword() uses SHA-256 to convert the plain password into a fixed-length hex string before it is ever sent to the database. Only this hash is stored in the PasswordHash column — the real password is never written to the database. During login, the entered password is hashed the same way and compared against the stored hash, so the comparison never happens on plain text. This matters because if the database were ever leaked or accessed by someone else, the actual passwords would still not be readable. For a production system, a slower password-specific algorithm with a per-user salt (such as bcrypt, Argon2id, or PBKDF2) would normally be preferred over a single unsalted SHA-256 pass, since SHA-256 alone is fast to brute-force at scale.

8. SQL Injection Demonstration

(a) Vulnerable code (demo only — not in the final submission)

A separate VulnerableDemoForm built the SQL command by concatenating the raw input directly into the query string:

```
string query = "SELECT * FROM dbo.Users WHERE Username='" + username +
               '" AND PasswordHash='" + password + "'";
```

Exploit input used: Username = irfan, Password = ' OR '1'='1

Result: the login succeeded with no valid password, because the injected text turned the WHERE clause into an always-true condition (see Figure 5).

(b) Fixed code (used in the actual project)

DatabaseHelper.ValidateLogin() never builds SQL by concatenating input. It uses a parameter for the username, and the password itself is never placed into the SQL string at all — it is hashed in code and compared against the stored hash after the row is fetched:

```
string query = "SELECT PasswordHash, FullName FROM dbo.Users WHERE Username = @username";
cmd.Parameters.AddWithValue("@username", username);
```

(c) Same input against the fixed version

The same style of input (username irfan, password ' OR '1'='1) was tried against the real LoginForm. Result: "Invalid username or password" — the injection no longer works (see Figure 6).

(d) Why parameters stop the attack

In a parameterized query, the value of @username is sent to SQL Server separately from the SQL command text, so SQL Server only ever treats it as a piece of data, never as part of the command to execute. Typing ' OR '1'='1 cannot "break out" of the query with a quote the way it does in the concatenated version — it is simply treated as a literal string (and, in the login path, hashed and compared like any other password), so it fails to match and the login is rejected normally.

9. Bonus Tasks Attempted

1. Centralized DatabaseHelper class

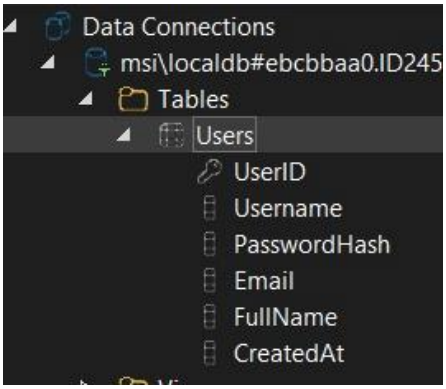
All database logic (connection handling, password hashing, registration, login validation, and fetching/searching users) lives in DatabaseHelper.cs. RegisterForm, LoginForm, and HomeForm never create a SqlConnection or SqlCommand directly — they only call DatabaseHelper methods.

2. Search/filter grid

DatabaseHelper.GetUsers() accepts an optional search term and, when supplied, adds a parameterized LIKE @term clause to filter by username. HomeForm's Search and Clear buttons call this method to filter the DataGridView shown on the Home screen.

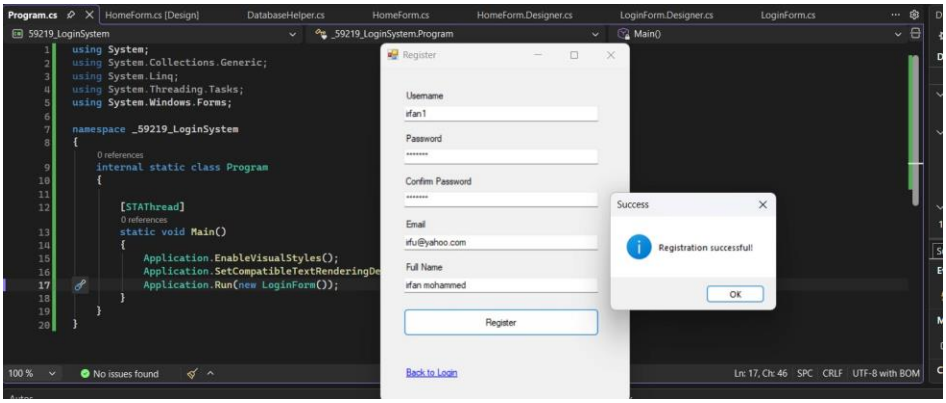
10. Required Screenshots

Figure 1 — Users table design (SQL Server Object Explorer)



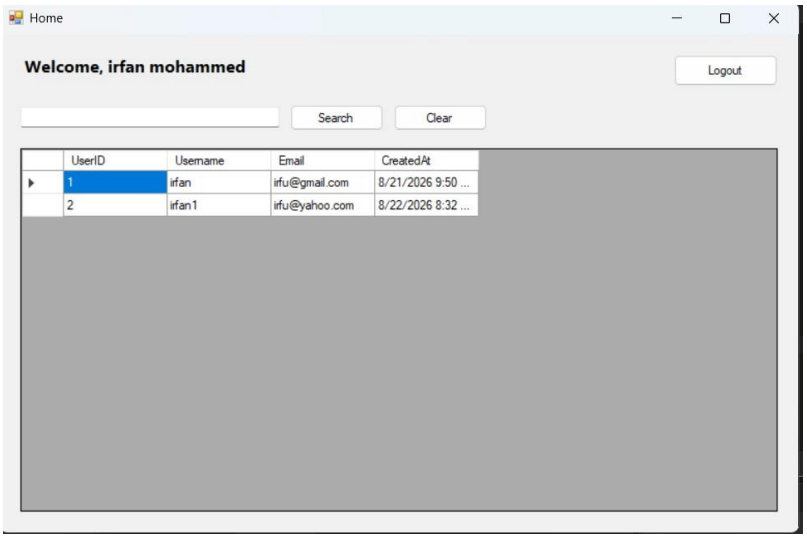
The Users table with its six columns: UserID, Username, PasswordHash, Email, FullName, CreatedAt.

Figure 2 — Successful registration



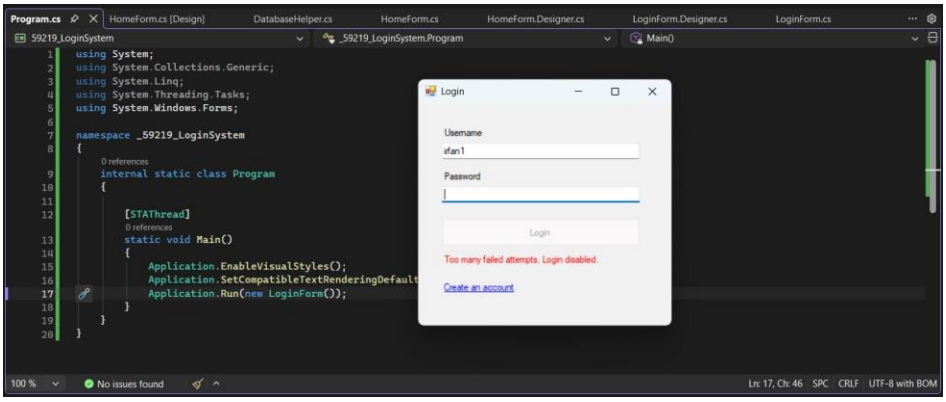
RegisterForm after a new account (irfan1) is created, showing the "Registration successful!" confirmation.

Figure 3 — Successful login with the users grid



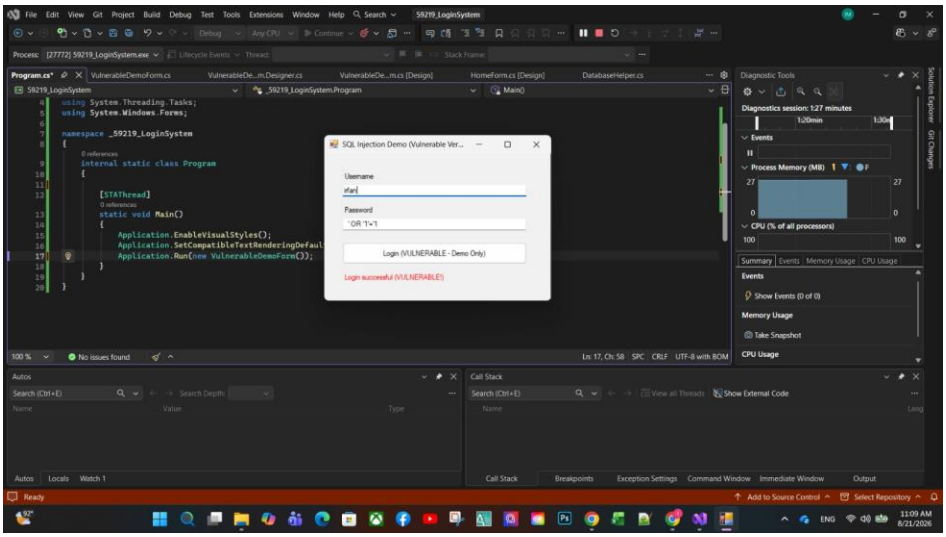
HomeForm after a successful login, showing the "Welcome, irfan mohammed" message and the DataGridView listing registered users.

Figure 4 — Login disabled after failed attempts



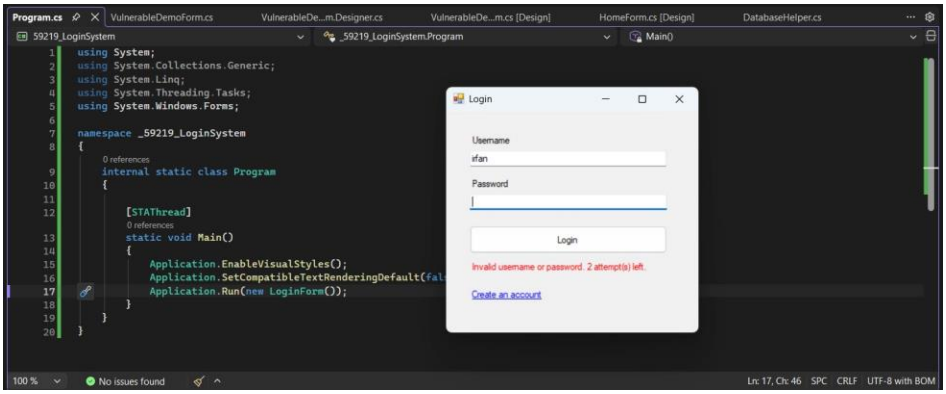
LoginForm after 3 consecutive failed attempts — the status message reads "Too many failed attempts. Login disabled." and the Login button is disabled.

Figure 5 — SQL injection succeeding on the vulnerable demo form



VulnerableDemoForm accepting username irfan with password ' OR '1'='1 — "Login successful (VULNERABLE!)" because the input was concatenated directly into the SQL string.

Figure 6 — Same injection attempt failing on the fixed LoginForm



The real, parameterized LoginForm rejecting the same style of injection input with "Invalid username or password. 2 attempt(s) left."

Note: a dedicated logout screenshot was not captured separately, since logout is a plain close-and-return-to-Login action with no additional UI state beyond the cleared LoginForm shown after Figure 4/5.

11. Problems Encountered and Solutions

Problem 1 — Toolbox showed no controls for RegisterForm

Cause: The Toolbox was opened while the code (.cs) view was active instead of the Designer view, so it showed "There are no usable controls in this group" with an empty panel.

Solution: Switched to Design view and reopened the Toolbox (View → Toolbox); Common Controls such as Button, TextBox, and Label then appeared normally.

Problem 2 — Build failed after adding ConfigurationManager

Cause: DatabaseHelper.cs used ConfigurationManager.ConnectionStrings[...] to read the connection string, but System.Configuration was not referenced by the project by default.

Solution: Added the reference manually via Project → Add Reference → Assemblies → Framework → System.Configuration.

Problem 3 — Could not add a SQL Server connection in Server Explorer

Cause: Visual Studio threw "Could not load file or assembly 'Microsoft.Data.SqlClient'... The system cannot find the file specified" because the "Data sources for SQL Server support" individual component was not installed.

Solution: Installed that component through the Visual Studio Installer, which resolved the connection error.

Problem 4 — Connection string pointed at the wrong SQL Server instance

Cause: The machine did not have a named SQL Server Express instance like localhost\SQLEXPRESS; only LocalDB ((localdb)\MSSQLLocalDB) was available.

Solution: Adjusted the App.config connection string to target (localdb)\MSSQLLocalDB instead of copying a sample connection string, and verified it with Test Connection in Server Explorer.

Problem 5 — Users table did not appear under Tables

Cause: The first Data Connections entry pointed at the master database instead of the new ID2459219_LoginDB database.

Solution: Created a new connection and explicitly selected ID2459219_LoginDB from the "Select or enter a database name" dropdown.

Problem 6 — "Call is ambiguous" build error on the injection demo form

Cause: While building the deliberately vulnerable demo form, a duplicate constructor ended up defined in both the Designer file and the code-behind file.

Solution: Fully replaced both VulnerableDemoForm.cs and VulnerableDemoForm.Designer.cs with clean versions so the constructor was only defined once.

Problem 7 — CS0103 errors after deleting the demo form

Cause: After capturing the injection-demo screenshots, only VulnerableDemoForm.Designer.cs was deleted, leaving VulnerableDemoForm.cs behind. Several CS0103 errors appeared (InitializeComponent, txtUsername, txtPassword, lblResult "does not exist in the current context") because those fields were only defined in the deleted Designer file.

Solution: Deleted the remaining VulnerableDemoForm.cs as well, which fixed the build.

12. SQL Query Safety

The production database methods — UsernameExists, RegisterUser, ValidateLogin, and GetUsers — all use parameterized SQL (SqlCommand.Parameters.AddWithValue) rather than string concatenation. The only intentionally concatenated SQL in the project was the separate vulnerable demo form used for the injection demonstration in Section 8, which was removed from the final build after the screenshots were taken.

13. Running the Project

- Open 59219_LoginSystem.slnx in Visual Studio 2022.
- Run Schema.sql against a LocalDB instance ((localdb)\MSSQLLocalDB) to create ID2459219_LoginDB and the Users table.
- Update App.config if a different SQL Server instance is used.
- Build and run the project.
- Register a new user, then log in with that account.

- Inspect the Home screen: the welcome message and the users grid, including Search/Clear filtering.
- Test logout and confirm the Login form reappears cleared.
- Test the login-failure lockout by entering a wrong password 3 times.

14. GitHub Submission

The repository is named 59219_LoginSystem. Before committing, bin/, obj/, and .vs/ are excluded via .gitignore. The repository contains README.md, Schema.sql, this report, and the required screenshots under Screenshots/. No real database passwords are committed, since the connection string relies on LocalDB integrated security.